TASK 8: Read and Print Data Safely

Name: Nagineni Sunil Kumar

Register Number: 192472330

1. Aim

To accept input safely, handle invalid input, and display fare and balance information in a readable format.

2. C++ Program

```
#include <iostream>
#include <iomanip>
#include <limits>
using namespace std;
int main() {
    int hour;
    long long balancePaise = 10000;
    cout << "Enter journey hour (0-23): ";
    while (!(cin >> hour) || hour < 0 || hour > 23) {
        cout << "Invalid hour. Enter a number from 0 to 23: ";
        if (cin.fail()) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
        }
    }
```

```
long long farePaise = 1000;
bool peak = (hour >= 8 && hour < 10) ||
            (hour >= 17 && hour < 20);
if (peak)
    farePaise = farePaise * 120 / 100;
if (balancePaise - farePaise < 5000) {
    cout << "Journey denied: low balance\n";
    return 0;  }
balancePaise -= farePaise;
cout << fixed << setprecision(2);
cout << "Journey Report\n";
cout << "Hour: " << hour << '\n';
cout << "Fare: Rs " << farePaise / 100 << '.'
     << setw(2) << setfill('0') << farePaise % 100
     << setfill(' ') << '\n';
cout << "Remaining balance: Rs "
     << balancePaise / 100 << '.'
     << setw(2) << setfill('0') << balancePaise % 100
     << setfill(' ') << '\n';

return 0; }
```

3. Sample Output — Valid Input

Enter journey hour (0-23): 9

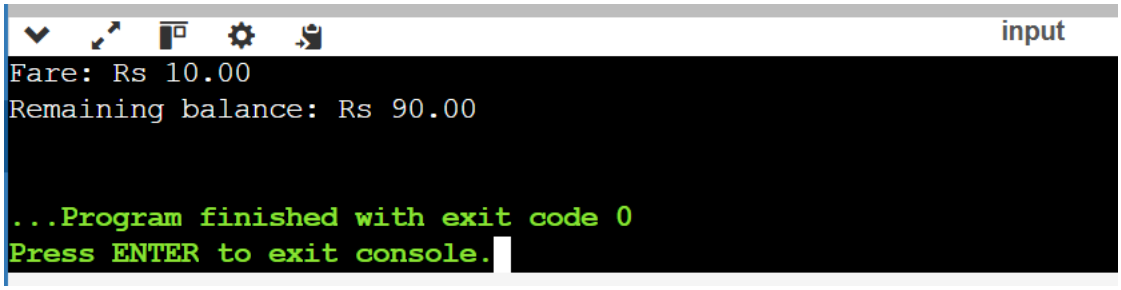
Journey Report

Hour: 9

Fare: Rs 12.00

Remaining balance: Rs 88.00

Obtained output :-



4. Sample Output — Invalid Input

Enter journey hour (0-23): abc

Invalid hour. Enter a number from 0 to 23: 9

Journey Report

Hour: 9

Fare: Rs 12.00

Remaining balance: Rs 88.00

5. Explanation

- cin >> hour reads the hour.
- cin.fail() detects an input failure.
- cin.clear() resets the stream error state.

- cin.ignore() discards the invalid input.
- setw(2) sets a minimum field width of two characters.
- setfill('0') adds a leading zero when needed.
- The program stores money in paise and formats it as rupees and paise when printing.

6. Alternative Concept

An alternative is to read a full line using getline() and convert it with stoi() inside a try-catch block. Both methods need to inspect the input, so processing a line of length (L) takes **O(L)** time. getline() gives more control over the full input line, while stream-based input requires clearing and discarding invalid input before reading again.

7. Complexity

The validation loop can repeat until valid input is supplied. For each successful input, fare calculation takes O(1) time and uses O(1) extra space.

8. Conclusion

Input validation prevents invalid hours from entering the fare calculation. Formatting functions improve readability, and recovery from failed input helps the gate program continue safely.